Project Rocky: Inverted Pendulum Balancing Robot

Enzo Smajlaj, David Barsoum, Jacob Keller

April 18, 2026

1 Introduction

The goal of this project is to balance the "Rocky" robot, which is a modified Balboa 32U4 Balancing Robot. The robot is modeled as an unstable inverted pendulum on a horizontally translating cart. An inverted pendulum is a pendulum that balances upside-down, meaning it has its mass at the top and is balanced on a pivot point below its center of gravity. It is a common problem in control theory to create a system to balance this inverted pendulum; through nested feedback control networks, we can stabilize the inherently unstable system to balance and reject external disturbances.

To create the control system to balance Rocky, we (1) determined the physical constants, including characterizing the motors and natural frequency of the system. (2) We determined control constants by finding poles to fit our desired response. (3) We created Simulink models in MATLAB to model the three-pole and five-pole systems. (4) We tested our system on a physical Rocky with the Arduino IDE.

2 Parameter Identification

To design an effective control system, we first determined the specific hardware parameters for Rocky's motors and pendulum dynamics.

2.1 Motor Parameter Estimation

Our first step is to characterize our motor constants. The goal of this is to develop a relationship between motor commands and the resulting velocity of Rocky's wheels to effectively command the motors to send appropriate speeds. The wheel velocity $V(s)$ in response to the motor command $Y(s)$ can be modeled as a first-order system. Setting the system gain as K and the time constant as τ, the transfer function is:

$$M(s) = \frac{V(s)}{Y(s)} = \frac{K/\tau}{s + 1/\tau} \tag{1}$$

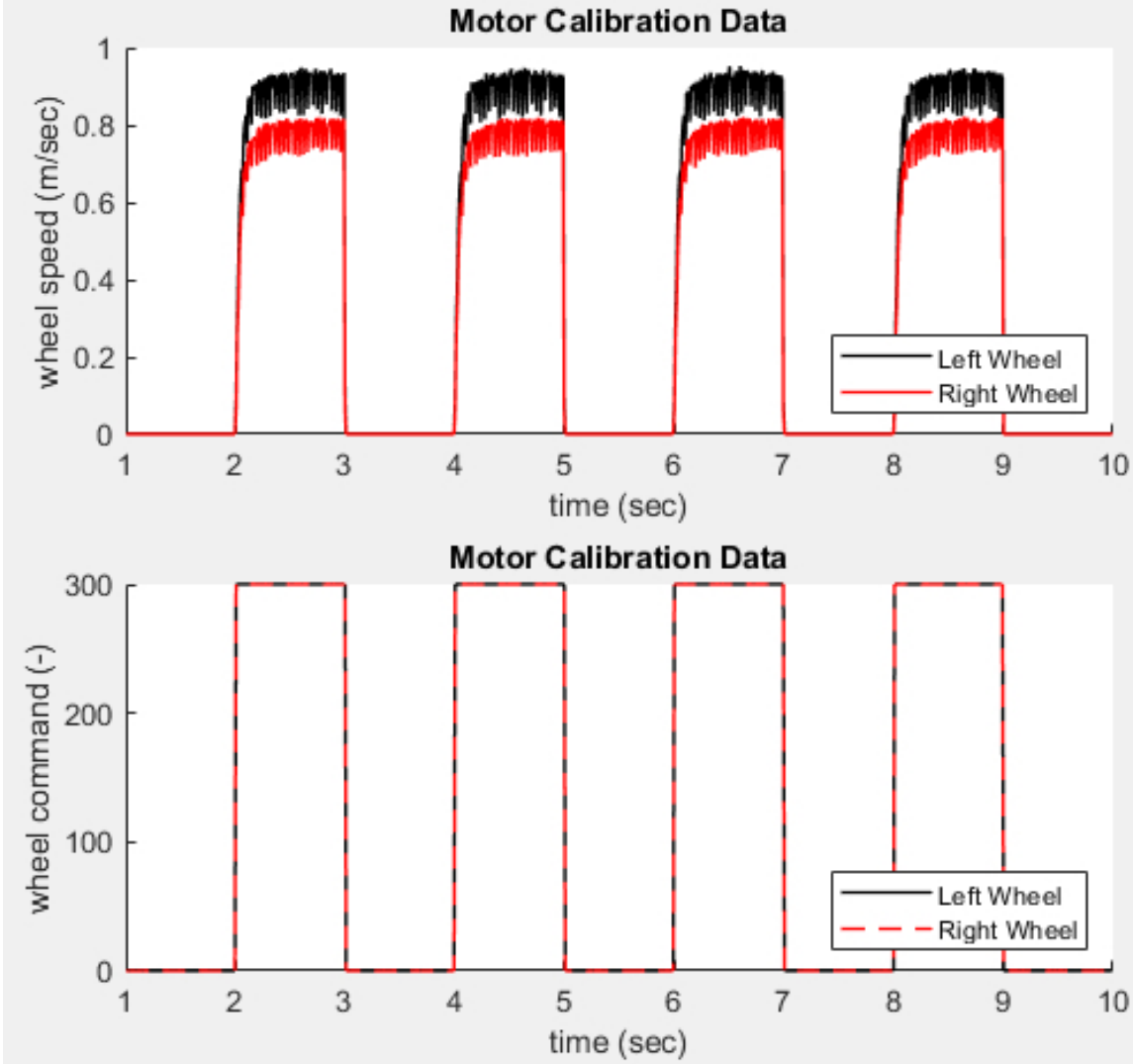

Figure 1: Comparison of motor command to measured wheel speed. A small, but noticable discrepancy is seen between the left and right wheels, although this was deemed negligible in the project.

To find the time-domain response to a constant commanded step input of magnitude $Y_{step} = 300$, we take the inverse Laplace transform of the product of the step input and the transfer function:

$$V(s) = \frac{300}{s} M(s) = \frac{300K/\tau}{s(s + 1/\tau)} \tag{2}$$

Using Laplace transform tables for a step response, we extract the analytical expression for the velocity over time:

$$\mathcal{L}^{-1}\{V(s)\} = 300\frac{K}{\tau}\mathcal{L}^{-1}\left\{\frac{1}{s(s+1/\tau)}\right\} = 300\frac{K}{\tau} \cdot \tau\left(1 - e^{-\frac{1}{\tau}t}\right) \tag{3}$$

$$v(t) = 300K(1 - e^{-t/\tau}) \tag{4}$$

We collected experimental data by sending a step of 300 and fit it to the exponential curve $v(t) = c(1 - e^{-at})$ using MATLAB's fit object method. Comparing our mathematical derivation to the curve fit equation, we see that $a = 1/\tau = \alpha$ and $c = 300K$, meaning the motor gain β (or K) is exactly $c/300$.

From our collected data, the curve fitter extracted the following coefficients:

- Exponential decay rate (a): 22.17
- Steady-state velocity (c): 0.9 m/s

Therefore, our calibrated motor parameters are:

$$\alpha = \mathbf{22.17} \text{ rad/s}, \quad \beta = \mathbf{0.003} \text{ (m/s)/command} \tag{5}$$

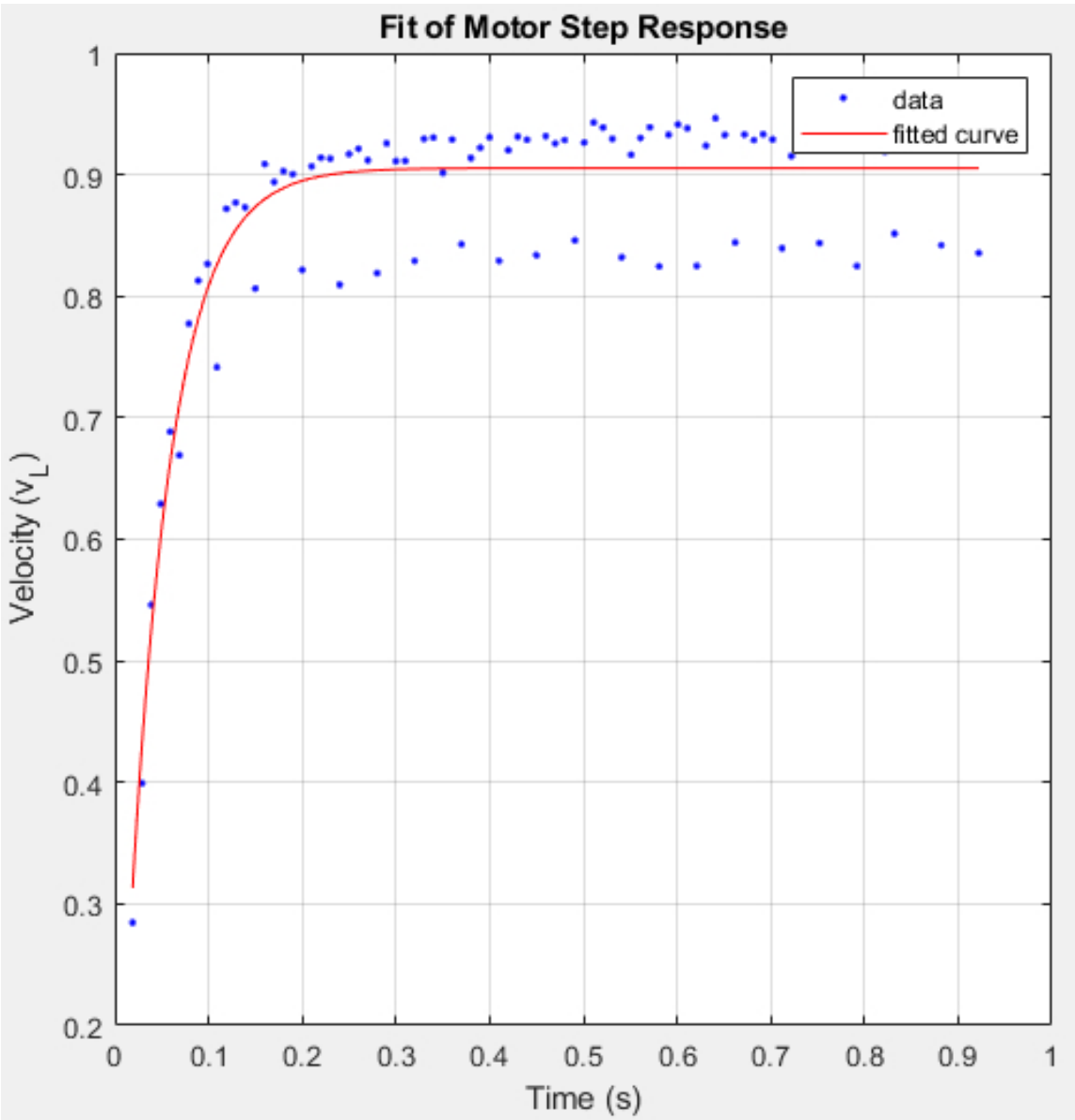

Figure 2: Wheel velocity over time for a single step input. Best fit line was found using a custom function with the curve fit object.

2.2 Pendulum Parameter Estimation

The behavior of our system depends upon the natural frequency of the inverse pendulum. We can find this value experimentally by holding Rocky while it swings back and forth, recording the IMU data for analysis, as this value is determined by the equation:

$$\omega_n^2 = g/l_{eff} \tag{6}$$

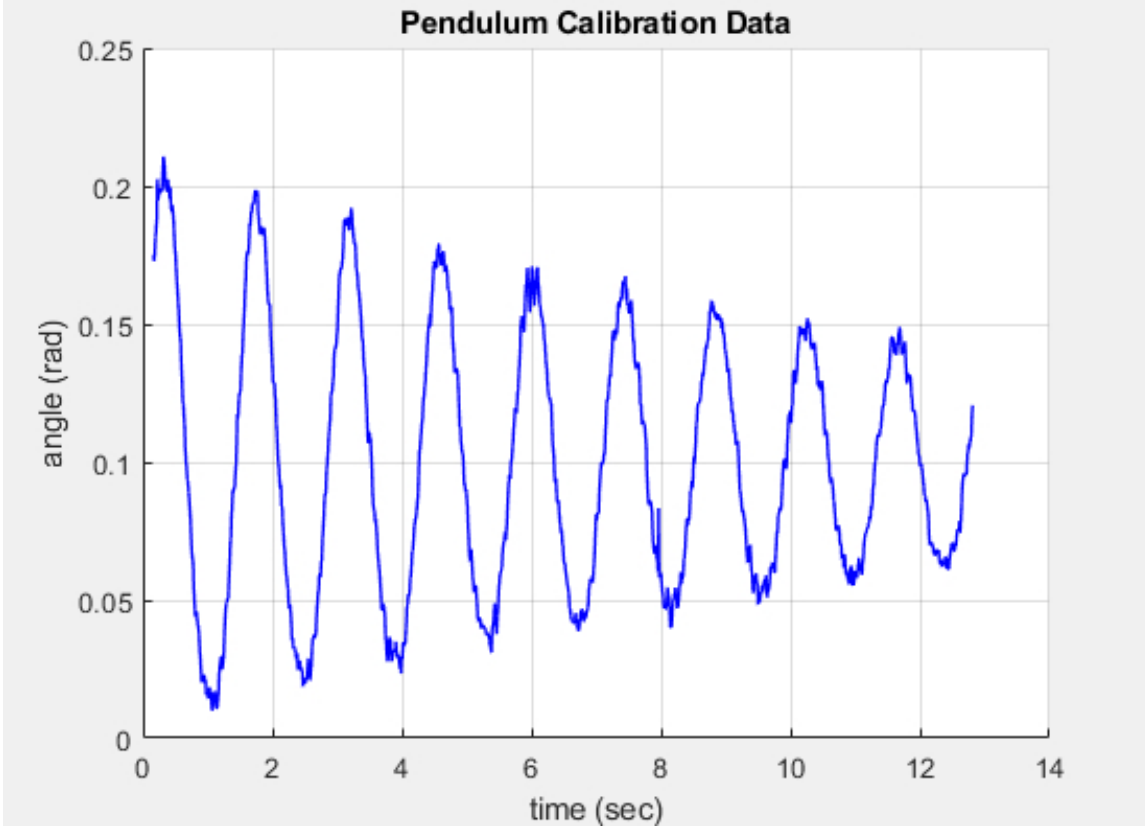

Figure 3: Measured angle over time. Data collected from the Rocky's internal IMU after being released and allowed to swing freely while being held from the top (non-inverted pendulum).

We recorded the free response of Rocky swinging back and forth, measuring the angle with respect to time via the internal IMU, and calculated the time interval between consecutive peaks to determine the period of oscillation, T. We found $T \approx 1.41$ seconds. The natural frequency is calculated as:

$$\omega_n = \frac{2\pi}{T} = \frac{2\pi}{1.41} = 4.44 \text{ rad/s} \tag{7}$$

Using the relationship for natural frequency and $g = 9.81$ m/s^2, we derived the effective length:

$$\mathbf{l_{eff}} = \frac{g}{\omega_n^2} = \frac{9.81}{(4.44)^2} = \mathbf{0.497} \text{ m} \tag{8}$$

All of the necesarry fitting and calculations to derive these values can be found in `calibrationfile.m`.

3 System Response

Using feedback from sensor information, we control the system's response to external stimuli, or disturbances, and introduce stability into the system. This is done via two main controllers: a 3-pole system that counteracts angular disturbances, and a 5-pole system that counteracts angular *and* positional disturbances. To implement these, we add feedback loops into our calculations, thus providing additional coefficients and modifying our characteristic equation. Analyzing our transfer function leads to the determination of our system's poles, which govern system behavior. Optimal behavior will keep the Rocky robot erect without tilting; however, we see that small levels of oscillation are beneficial in the stability of our system, thus enabling a quick response time without extreme overshoot. The following procedures cover the systematic approach to determining the optimal pole values to maintain a stable system. matlab

3.1 3-Pole System

The initial goal is to balance the robot vertically and purely reject angular disturbances using a single feedback loop.

3.1.1 Controller Architecture & Derivation

The dynamics of the inverted pendulum relate the angle $\Theta(s)$ to the cart velocity $V(s)$. Linearizing the equation of motion for small angles yields the transfer function $H_{v\theta}(s) = \frac{s/l_{eff}}{s^2-\omega_n^2}$. To stabilize it, we apply a PI controller, $K(s) = K_p + \frac{K_i}{s}$, in a negative feedback loop with the motor $M(s) = \frac{\alpha\beta}{s+\alpha}$.

We model a disturbance $D(s)$ injected directly into the angle output. By setting the desired reference angle to zero ($\Theta_d(s) = 0$), the error signal becomes $E(s) = -\Theta(s)$. The output angle is the sum of the disturbance and the forward path:

$$\Theta(s) = D(s) + H_{v\theta}(s)M(s)K(s)E(s) = D(s) - H_{v\theta}(s)M(s)K(s)\Theta(s) \tag{9}$$

Factoring out $\Theta(s)$ and using Black's formula, the closed-loop transfer function relating the angle to the disturbance is:

$$\frac{\Theta(s)}{D(s)} = \frac{1}{1 + K(s)M(s)H_{v\theta}(s)} \tag{10}$$

Substituting our specific transfer functions:

$$\frac{\Theta(s)}{D(s)} = \frac{1}{1 + \left(\frac{K_p s + K_i}{s}\right)\left(\frac{\alpha\beta}{s+\alpha}\right)\left(\frac{s/l_{eff}}{s^2-\omega_n^2}\right)} \tag{11}$$

To simplify, we multiply the numerator and denominator by the denominator of the feedback term, $s(s+\alpha)(s^2-\omega_n^2)$:

$$\frac{\Theta(s)}{D(s)} = \frac{s(s+\alpha)(s^2-\omega_n^2)}{s(s+\alpha)(s^2-\omega_n^2) + (K_p s + K_i)(\alpha\beta)(s/l_{eff})} \tag{12}$$

The s term cancels from the numerator and denominator:

$$\frac{\Theta(s)}{D(s)} = \frac{(s+\alpha)(s^2-\omega_n^2)}{(s+\alpha)(s^2-\omega_n^2) + (K_p s + K_i)\left(\frac{\alpha\beta}{l_{eff}}\right)} \tag{13}$$

Expanding the polynomial in the denominator gives us our final closed-loop transfer function and characteristic equation:

$$\frac{\Theta(s)}{D(s)} = \frac{(s+\alpha)(s^2-\omega_n^2)}{s^3 + \alpha s^2 + \left(K_p\frac{\alpha\beta}{l_{eff}} - \omega_n^2\right)s + \left(K_i\frac{\alpha\beta}{l_{eff}} - \alpha\omega_n^2\right)} \tag{14}$$

3.1.2 Pole Derivation

Given the nature of the characteristic polynomial, embodying the structure of a trinomial, it may be written as the product of three factors. Expanded, this form looks like:

$$(s-p_1)(s-p_2)(s-p_3) = s^3 - (p_1+p_2+p_3)s^2 + (p_1p_2+p_2p_3+p_1p_3)s - p_1p_2p_3 \tag{15}$$

By comparing this to the actual system denominator, we identify the following coefficient identities:

$$p_1 + p_2 + p_3 = -\alpha \tag{16}$$

$$p_1p_2 + p_2p_3 + p_1p_3 = \frac{K_p\alpha\beta}{l_{\text{eff}}} - \omega_n^2 \tag{17}$$

$$p_1p_2p_3 = \alpha\omega_n^2 - \frac{K_i\alpha\beta}{l_{\text{eff}}} \tag{18}$$

Therefore:

$$p_1 + p_2 + p_3 = -\alpha = -22.17 \tag{19}$$

Mathematically, this implies a constraint on the sum of the poles, defining the average of the poles as $\alpha/3$. It is known that the pole closest to the imaginary axis (the least negative pole) is the dominant pole, as its exponential term e^{pt} decays the slowest and therefore dominates the overall settling time of the system.

Since the average is fixed at $\alpha/3$, or 7.39, it is mathematically ideal to place all three poles (or the real parts of complex poles) exactly at this average. If we were to make one pole smaller (e.g., pushing it further left to -10), the remaining poles would inherently have to become bigger (pushed closer to the imaginary axis) to maintain the average. The pole pushed to the right would immediately become the new dominant pole, resulting in a slower overall decay rate and having the opposite effect of our intention. Therefore, setting all real parts to exactly $-\alpha/3$ guarantees the absolute fastest worst-case decay rate.

Given these restraints, we can use some coefficient matching to determine our gain values:

$$\frac{K_p \alpha\beta}{l_{\text{eff}}} - \omega_n^2 = 3\left(\frac{\alpha}{3}\right)^2 = \frac{\alpha^2}{3} \quad \text{and} \quad \alpha\omega_n^2 - \frac{K_i \alpha\beta}{l_{\text{eff}}} = \left(-\frac{\alpha}{3}\right)^3 = -\frac{\alpha^3}{27}$$

Rearranging leads to:

$$K_p = \frac{l_{\text{eff}}}{\alpha\beta}\left(\frac{\alpha^2}{3} + \omega_n^2\right) \quad \text{and} \quad K_i = \frac{l_{\text{eff}}}{\alpha\beta}\left(\frac{\alpha^3}{27} + \alpha\omega_n^2\right) \tag{20}$$

Plugging in our derived values from earlier provides:

$$\mathbf{K_p = 1.3716 \times 10^3}, \qquad \mathbf{K_i = 6.2817 \times 10^3} \tag{21}$$

3.1.3 Simulink Model & Performance

We constructed and visualized a Simulink Model for the 3-pole system to determine the performance of our system under a given disturbance.

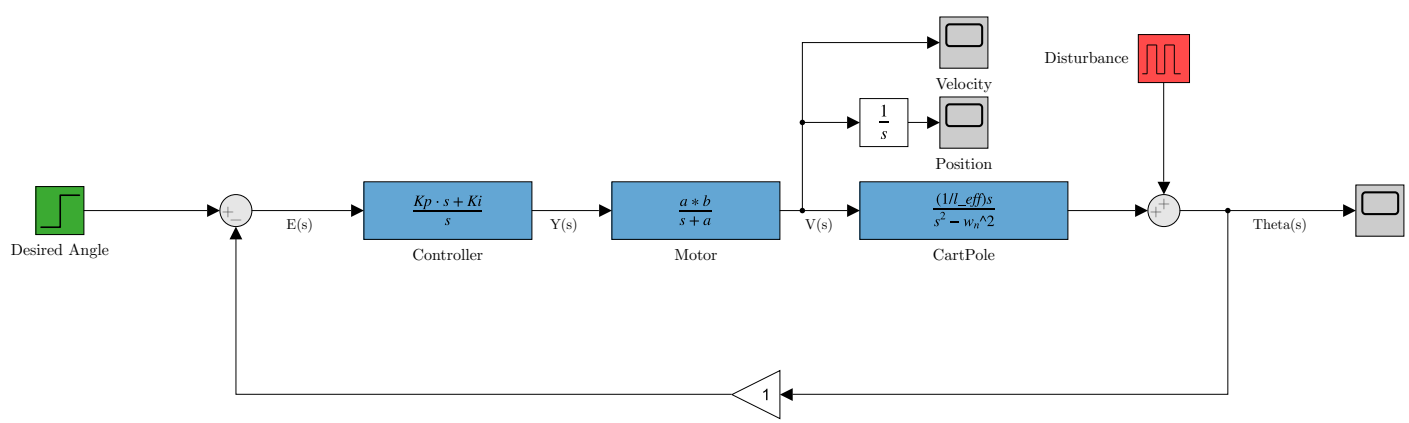

Figure 4: Simulink Block Diagram for the 3-Pole System

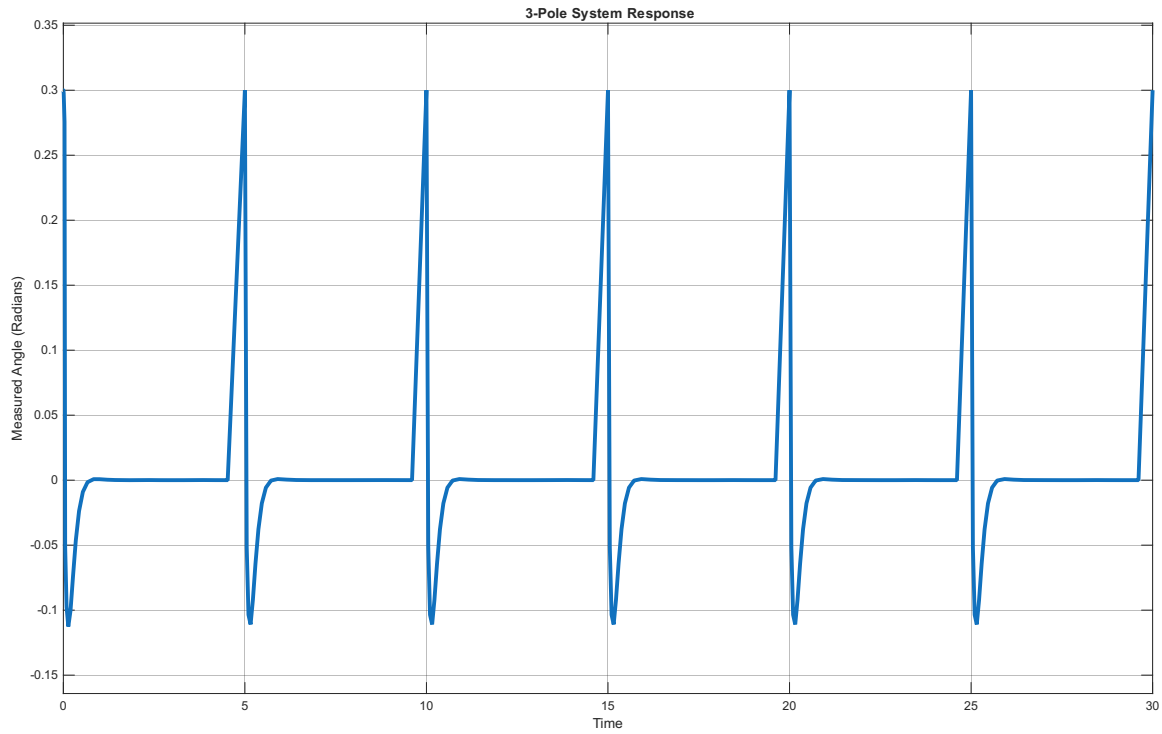

Figure 5: Scope output showing measured angle over time for the 3-pole Rocky system. The measured angle oscillates once then quickly returns to the 0 in response to each disturbance. This aligns with the observed behavior of our Rocky.

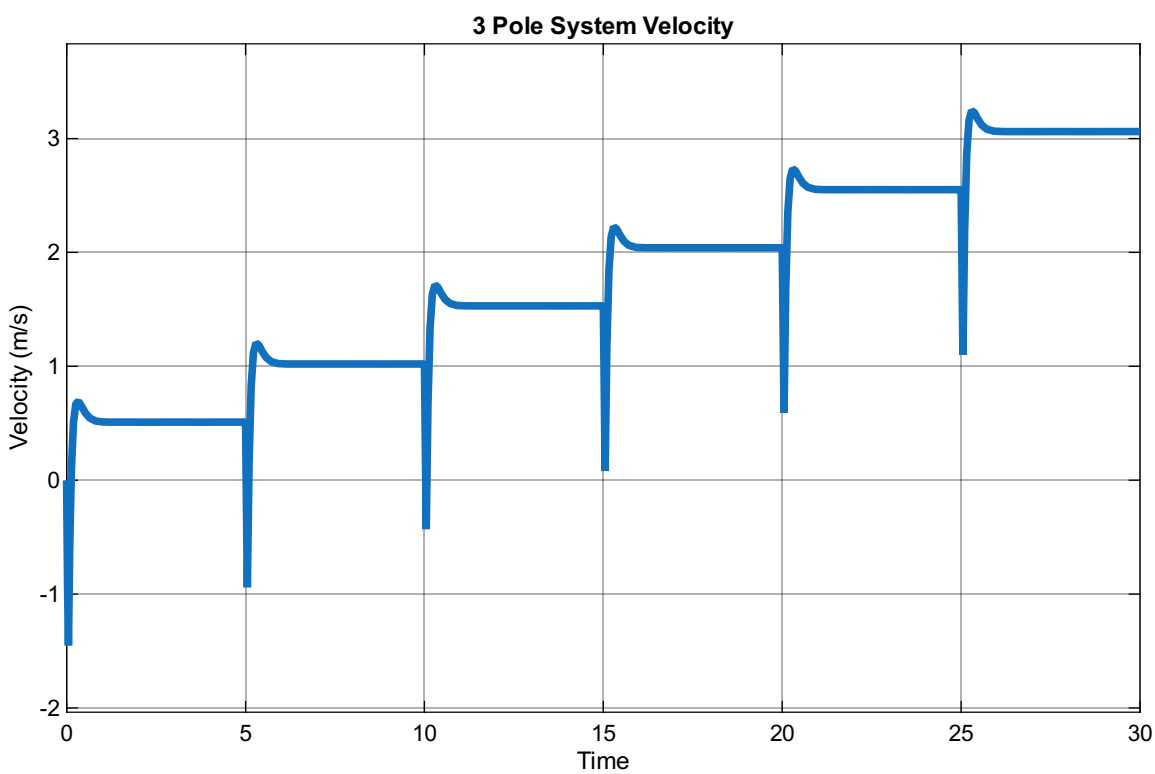

Figure 6: Scope output showing measured velocity over time for the 3-pole Rocky system. For each disturbance, the velocity dips and rises, before settling on a higher value, as the robot adjusts. Over time, the velocity will continue to increase.

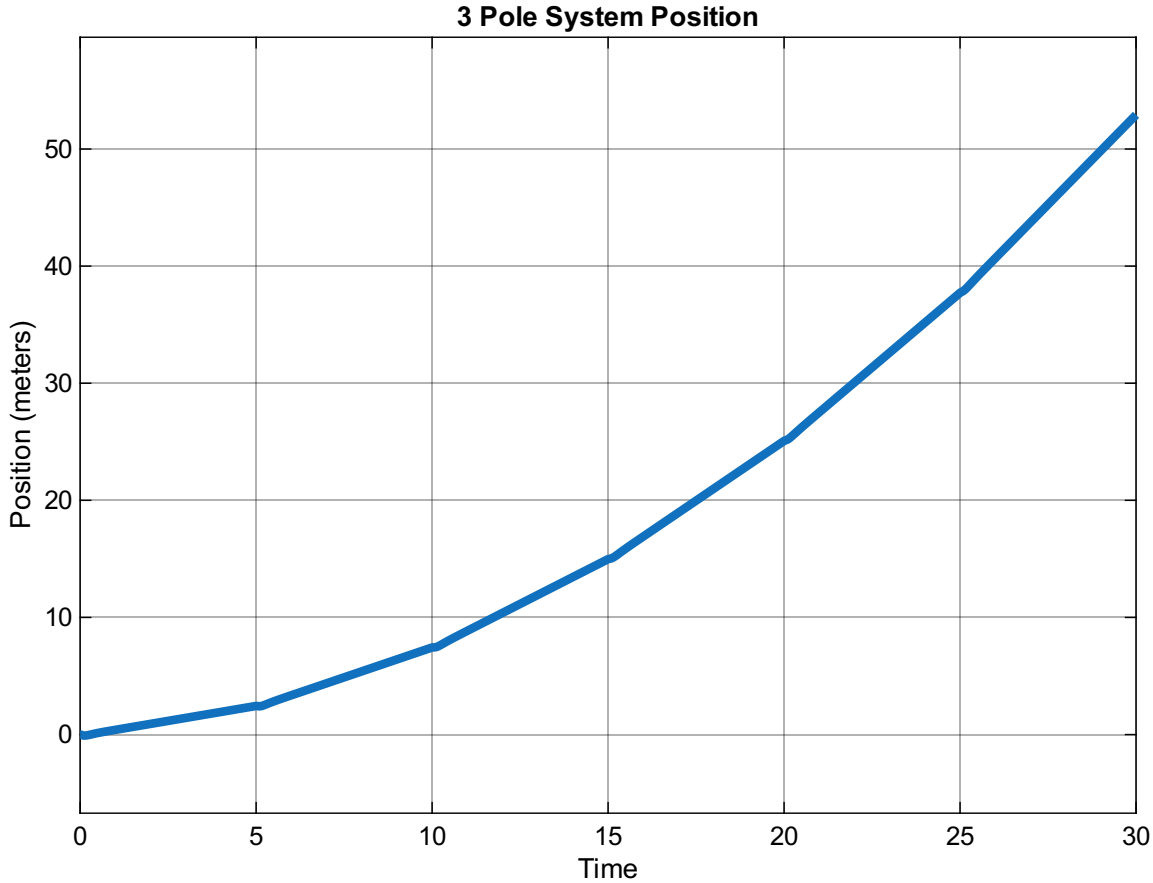

Figure 7: Scope output showing measured position over time for the 3-pole Rocky system. Over time, the position increases exponentially, as the velocity continues to speed up. In our testing, our Rocky would encounter obstacles before the position would be able to climb quite so high, but in an ideal situation, it would continue to move further and further away from the starting point.

3.2 5-Pole System

We then designed a nested feedback loop to simultaneously control the angle and regulate the cart's horizontal position to $x = 0$. This would prevent suboptimal situations where the Rocky may restore to a valid angle while continuously moving slightly forward or backwards, i.e. position sky-rocketing.

3.2.1 Controller Architecture & Derivation

We wrapped the inner motor loop with an outer PID-esque velocity controller. This takes the form:

$$Y(s) = \left(J_p + \frac{J_i}{s} + \frac{C_i}{s^2}\right)(V_r(s) - V(s)) \tag{22}$$

In the time domain, this controller outputs a reference motor command based on the difference between the reference velocity/position and the actual velocity/position. This looks like:

$$y(t) = -J_p \Delta v(t) - J_i \Delta x(t) - C_i \int \Delta x(t) dt \tag{23}$$

**For our programmatic implementations, all integral evaluations were approximated via Riemann sums.*

The introduction of this additional loop in our control modifies the characteristic polynomial to become fifth-degree. Consequently, symbolic derivations of the poles becomes difficult; however, there are still physical limitations of the robot and system designs that influence our pole values.

3.2.2 Pole Derivation

The process of deriving the poles necessary to stabilize the system under a disturbance involved repeated simulated tests with certain guidelines and expectations. This involved a baseline of value described here:

1. The response of the system must not be so extreme so as to saturate the possible motor speeds. This was defined earlier through our motor calibration, providing a value to remain under for given simulations via our SimuLink model.

2. The response of the system should be relatively quick so as to overcome the constant forces acting on the robot, i.e. rotating quickly enough to overcome the increasing force of gravity causing Rocky to fall.

3. The response of the system should not be fragile, similar to the extreme amplitude precaution, meaning it should not rely on small sensor readings, in the form of noise, to provide change.

Ultimately, we must provide poles that, although they may not be optimal, perform under real world conditions. As such, despite control systems performing optimally under critically damped conditions, we selected for our system to have slight oscillations; thus, we force a complex conjugate pair with non-zero imaginary values.

This process of extracting parameter values is done through the file `Rocky_5_closed_loop_poles.m`, where we supply poles and get the parameter values. We started our estimation based on theoretical ω_d and ζ values, where ζ is a value chosen based on our desired oscillation and ω_d is based on ζ and ω_n. With this, we see that our real components should be in the range of roughly $[-8, -2]$ for our dominant poles, and the imaginary portion will be slightly slower. Given this, our other poles will be entirely real but have different magnitudes so as to effect the system differently.

With this process continuously empirically tested through the file and the Rocky itself, we derived to the pole values of

$$\begin{aligned} p_1 &= -3.57 + 2.12i \\ p_2 &= -3.57 - 2.12i \\ p_3 &= -6 \\ p_4 &= -6.7 \\ p_5 &= -12 \end{aligned}$$

This yielded the following physically realizable control gains:

Control Loop	**Parameter Values**
Inner Loop	$K_p = 5.5942 \times 10^3, \quad K_i = 2.4907 \times 10^4$
Outer Loop	$J_p = 145.3917, \quad J_i = -5.1528 \times 10^3, \quad C_i = -6.3347 \times 10^3$

3.2.3 Simulink Model & Performance

Similarly, we constructed and visualized a Simulink Model for the 5-pole system to determine the performance under a given disturbance.

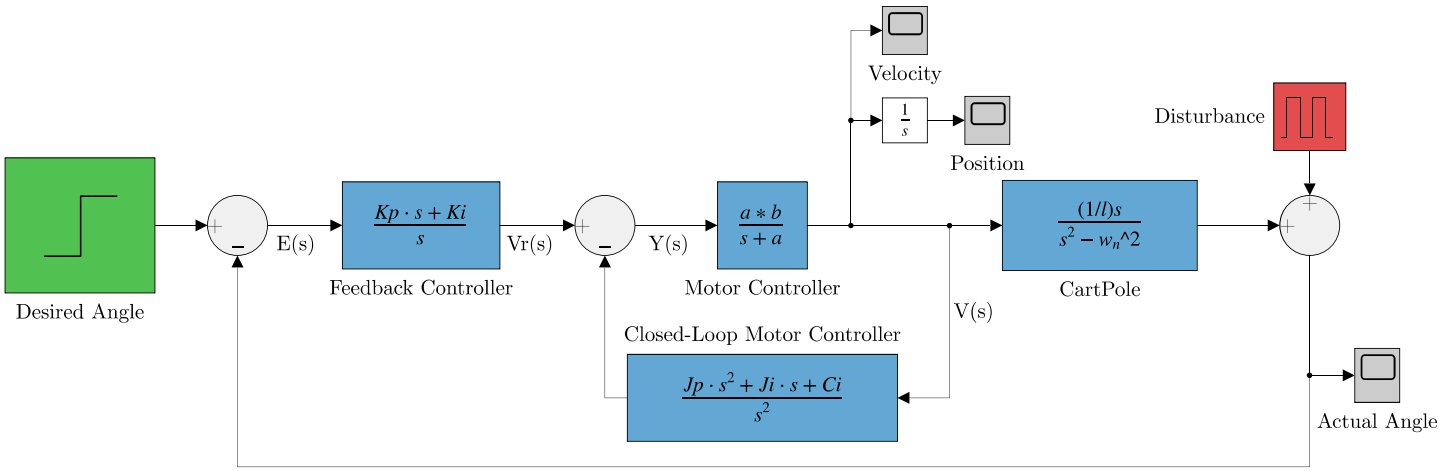

Figure 8: 5-Pole System Model, where each variable utilizes the previously derived values. Inputs are green, transfer functions are blue, and disturbances are red.

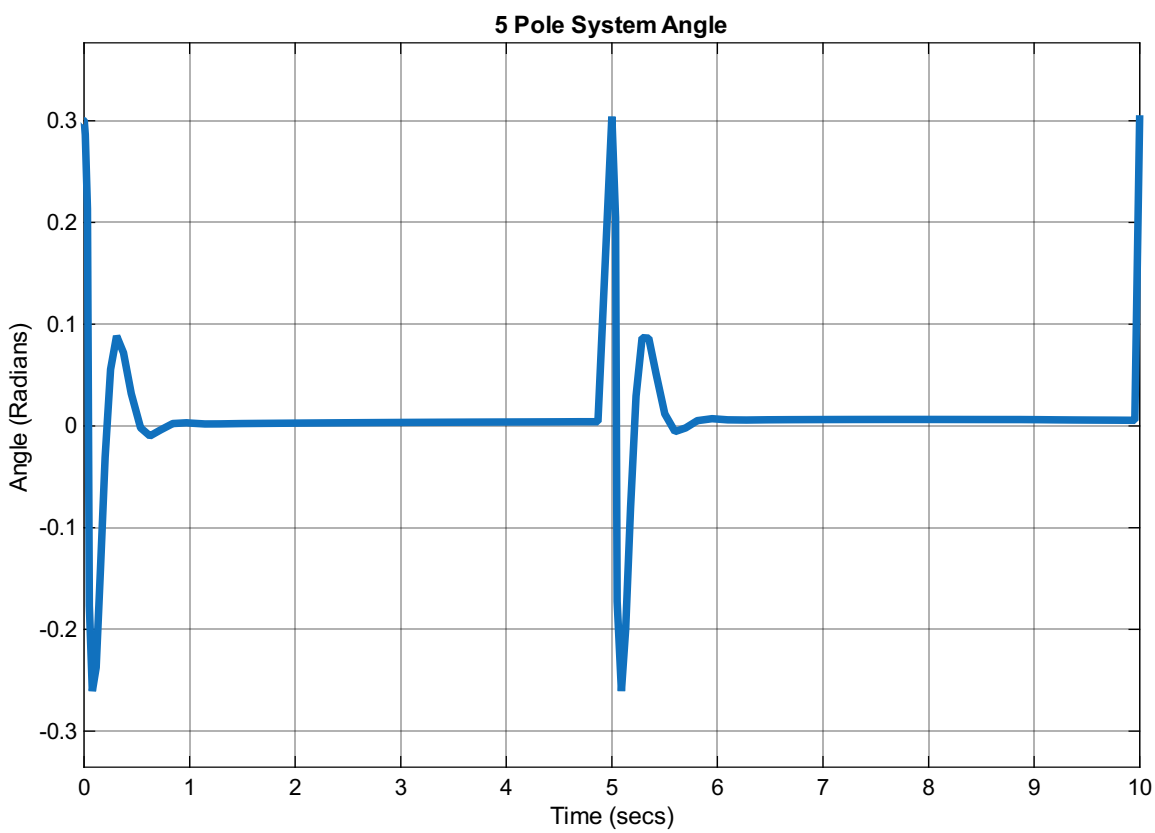

Figure 9: Scope output showing measured angle over time for the 5-pole Rocky system. There is slight overshoot in its response, but this was necessary to overcome physical imbalances, i.e. imperfect grip between the wheels and floor and other imperfect systems present.

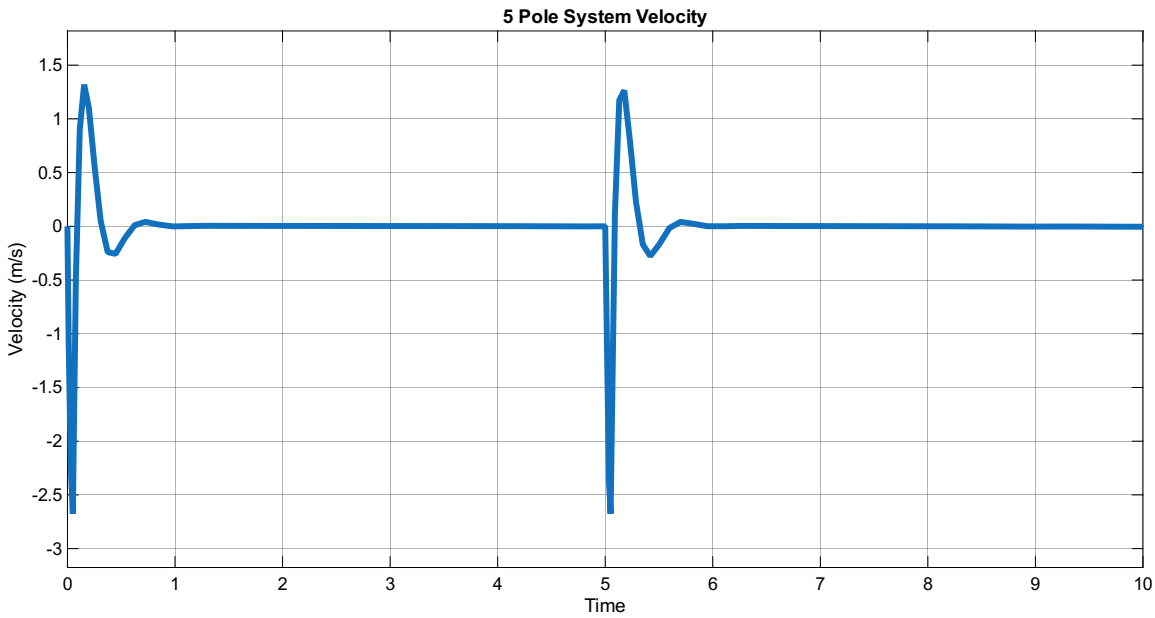

Figure 10: Scope output showing measured velocity over time for the 5-pole Rocky system. For each disturbance, the velocity oscillates as the Rocky corrects the pendulum angle, before decaying back to 0.

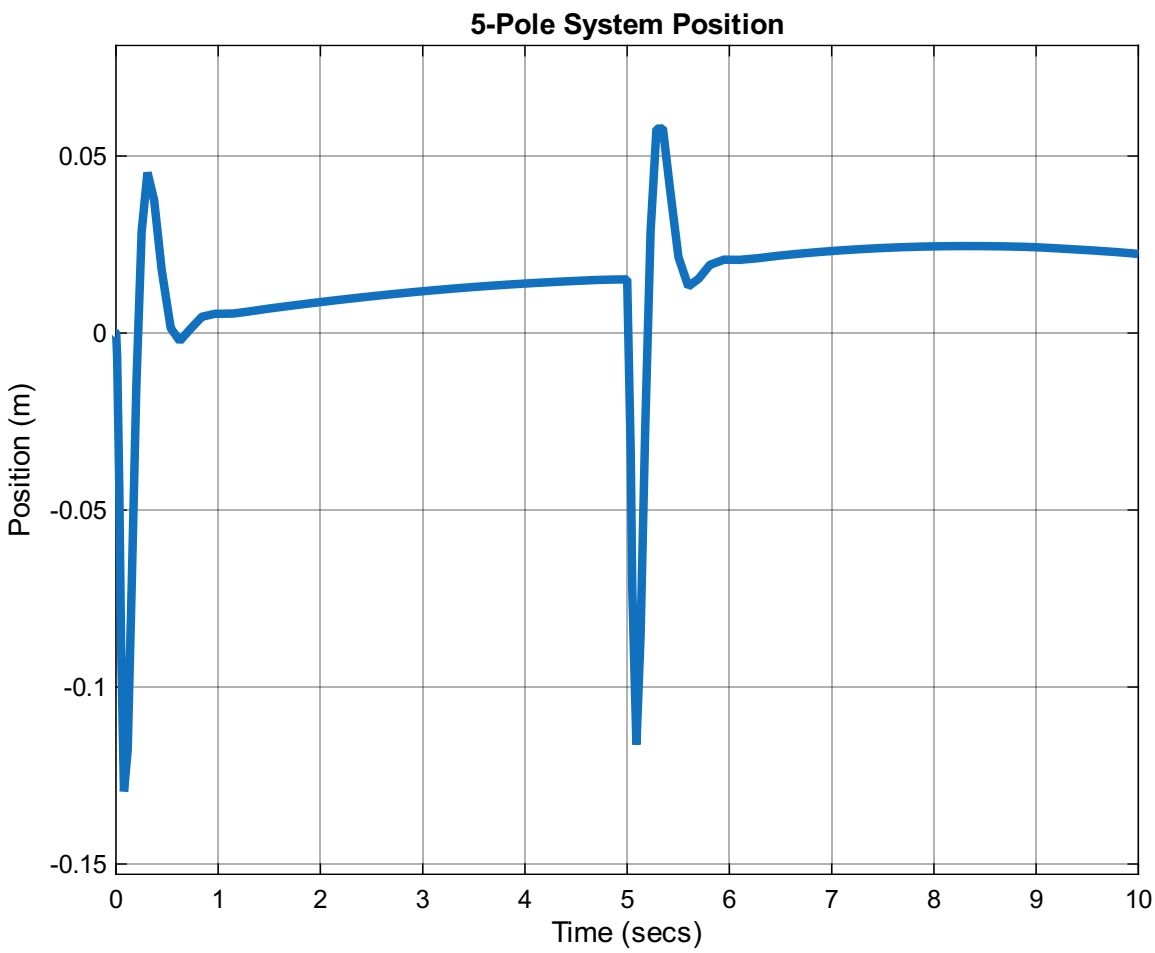

Figure 11: Scope output showing measured position over time for the 5-pole Rocky system. There exists a long-term oscillation around the position x=0, though given a single disturbance, the Rocky displays decay towards 0. For our purposes, this was deemed satisfactory when deployed on the robot's hardware. This long-term oscillation around 0 matched the observed behavior of our Rocky when testing.

4 Conclusion

By abstracting the Rocky robot into LTI subsystems (actuator dynamics and linearized cart-pole dynamics), we successfully derived feedback control systems capable of stabilizing an inherently unstable physical system. The addition of an outer position-tracking loop allowed the robot to reject disturbances and return to its origin. Although our behavior may now have been analytically ideal, it proved to be sufficient in balancing our robot.

4.1 Video Implementation

A video demonstrating the 3-pole balancing and 5-pole stationary balancing can be viewed here: `https://www.youtube.com/shorts/8q3cp3xEcFg`

5 Appendix

All files (including example code) for this project can be found in this drive: `https://drive.google.com/drive/folders/1awvFNSKUVdW3KW9CiYZDBz_AgoEKbhKl?usp=sharing` The following files were modified by us:

5.1 Calibration Script

The following is the file used to calibrate the motors and pendulum:

```
clear; clc;
%loads and plots the motor calibration data
%path and file name of data
fpath = 'C:\Users\esmajlaj\Downloads\'; %path (change this!)
fname_in = 'motor_data_inshallah.txt'; %file name (change this!)
%load the motor calibration data
motor_data = importdata([fpath,fname_in]);
%unpack the motor calibration data
t = motor_data(:,1);
y_L = motor_data(:,2); v_L = motor_data(:,3);
y_R = motor_data(:,4); v_R = motor_data(:,5);


%plot the motor calibration data
figure(1);
subplot(2,1,1);
hold on
plot(t,v_L,'k','linewidth',1);
plot(t,v_R,'r','linewidth',1);
xlabel('time (sec)'); ylabel('wheel speed (m/sec)');
title('Motor Calibration Data');
h1 = legend('Left Wheel','Right Wheel');
set(h1,'location','southeast');
subplot(2,1,2);
hold on
plot(t,y_L,'k','linewidth',1);
plot(t,y_R,'r--','linewidth',1);
xlabel('time (sec)'); ylabel('wheel command (-)');
title('Motor Calibration Data');
h2 = legend('Left Wheel','Right Wheel');
set(h2,'location','southeast');

%crop the data
start_i = 700;
end_i = 790;
v_l_cropped = v_L(start_i:end_i);
time_cropped = t(start_i:end_i) - 8;
ft = fittype('c * (1 - exp(-a * x))', 'independent', 'x', 'dependent', 'y');
opts = fitoptions('Method', 'NonlinearLeastSquares');
opts.StartPoint = [1, 14];

```

```

[fitresult, gof] = fit(time_cropped, v_l_cropped, ft, opts);
coeffs = coeffvalues(fitresult);
c_val = coeffs(2);
a_val = coeffs(1);

fprintf('Estimated b: %.2f\n', c_val / 300);
fprintf('Estimated a: %.2f\n', a_val);


figure;
plot(fitresult, time_cropped, v_l_cropped);
grid on;
xlabel('Time (s)');
ylabel('Velocity (v_L)');
title('Fit of Motor Step Response');
%%
clear;
%loads and plots the pendulum calibration data
%path and file name of data
fpath = 'C:\Users\esmajlaj\Downloads\';
fname_in = 'pendulum_data_mashallah.txt';
%load the pendulum calibration data
pendulum_data = importdata([fpath,fname_in]);
%unpack the pendulum calibration data
t = pendulum_data(:,1) - 103; theta = pendulum_data(:,2);
%plot the motor calibration data
figure(1);
hold on
plot(t,theta,'b','linewidth',1);
grid on;
xlabel('time (sec)'); ylabel('angle (rad)');
title('Pendulum Calibration Data');

%crop the data
theta_cropped = theta(190:end);
time_cropped = t(190:end);

regress_underdamped_response(time_cropped, theta_cropped)

w_n = 4.44;

l_eff = 9.81/(w_n)^2;
```

5.2 Controller Parameter Extraction Script

The following is the file used to, given poles, visualize system behavior and extract parameter values:

```
% Rocky_5_closed_loop_poles.m
%
% 1) Symbolically calculates closed loop transfer function of PI disturbannce
% rejection control system for Rocky.
% No motor model (M =1). With motor model (1st order TF)
%
% 2) Specify location of (target)poles based on desired reponse. The number of
% poles = denominator polynomial of closed loop TF
%
% 3) Extract the closed loop denomiator poly and set = polynomial of target
% poles
%
% 4) Solve for Ki, Kp, Ji, Jp, Ci to match coefficients of polynomials. In general,
% this will be underdefined and will not be able to place poles in exact
% locations. In this case (5th order), the control constants can be found exactly
%
% 5) Plot impulse response to see closed-loop behavior.
```

```
%
% based on code by SG. last modified 3/8/22 CL

clear all;
close all;

syms s a b l g Kp Ki Jp Ji Ci   % define symbolic variables

Hvtheta = -s/l/(s^2-g/l);       % TF from velocity to angle of pendulum

K = Kp + Ki/s;                  % TF of the PI angle controller
M = a*b/(s+a);                  % TF of motor (1st order model)
% M = 1;                        % TF without motor
%
J = Jp + Ji/s + Ci/s^2;         % TF of controller around motor-combined PI of x and v
Mfb = M/(1+M*J);                % Black's formula to get tf for motor with PI feedback control

%
% closed loop transfer function from disturbance d(t)totheta(t)
% Hcloop = 1/(1-Hvtheta*M*K)    % use this for no motor feedback
% with motor feedback
Hcloop = 1/(1-Hvtheta*Mfb*K)    % use this for motor with feedback

pretty(simplify(Hcloop))       % to display the total transfer function

% Substitute parameters and solve
% system parameters
g = 9.81;
l = .497;  %effective length
a = 22.17;           %nominal motor parameters
b = 0.003;        %nominal motor parameters

Hcloop_sub = subs(Hcloop) % sub parameter values into Hcloop

% specify locations of the target poles,
% choose # based on order of Htot denominator
% e.g., want some oscillations, want fast decay, etc.

% p1 = -1 + 2*pi*i    % dominant pole pair
% p2 = -1 - 2*pi*i    % dominant pole pair
% p3 = -10
% p4 = -8x
% p5 = -8.

p1 = -3.57 + 2.12*i   % dominant pole pair
p2 = -3.57 -2.12*i    % dominant pole pair
p3 = -6
p4 = -6.7    % dominant pole pair
p5 = -12    % dominant pole pair

% target characteristic polynomial
% if motor model (TF) is added, order of polynomial will increases
% tgt_char_poly = (s-p1)*(s-p2)*(s-p3)

% check polynomial-expand to fifth order
tgt_char_poly = (s-p1)*(s-p2)*(s-p3)*(s-p4)*(s-p5)
exp_tgt_char_poly = expand(tgt_char_poly)

% get the denominator from Hcloop_sub
[n d] = numden(Hcloop_sub)

% find the coefficients of the denominator polynomial TF
coeffs_denom = coeffs(d, s)

% divide though the coefficient of the highest power term
```

```
coeffs_denom = coeffs(d, s)/(coeffs_denom(end))
% num_coeff_denom = length(coeffs_denom)

% find coefficients of the target charecteristic polynomial
coeffs_tgt = coeffs(tgt_char_poly, s)
% num_coeff_tgt = length(coeffs_tgt)

% for check. reorder the coefficients to match the denomimator polynomial
for ii = 1:length(coeffs_denom)
    reord_coeffs_tgt(ii) = coeffs_tgt(length(coeffs_tgt) + 1 - ii);
end
% check roots of target polynomial-should be same as selected poles
roots_target = vpa(roots(reord_coeffs_tgt),4)


% solve the system of equations setting the coefficients of the
% polynomial in the target to the actual polynomials
solutions = solve(coeffs_denom(1:5) == coeffs_tgt(1:5), Jp, Ji,  Kp, Ki, Ci);

% display the solutions as double precision numbers
Kp = double(solutions.Kp)
Ki = double(solutions.Ki)
Ji = double(solutions.Ji)
Jp = double(solutions.Jp)
Ci = double(solutions.Ci)

%write out denominator polynomial
aaa = vpa(subs(coeffs_denom),4)

% reorder coefficients for the check polynomial
for ii = 1:length(coeffs_denom)
    chk_coeffs_denom(ii) = coeffs_denom(length(coeffs_denom) + 1 - ii);
end

% check poles should be same as chosen input poles
check_closed_loop_poles = vpa (roots(subs(chk_coeffs_denom)), 4)

% write out target polynomial
% bbb = vpa( expand( (s-check_closed_loop_poles(1))*(s-check_closed_loop_poles(2)) ...
%     *(s-check_closed_loop_poles(3))*(s-check_closed_loop_poles(4)) ...
%     *(s-check_closed_loop_poles(5)) ) )



% Plot impulse and step responses of closed-loop system
    TFstring = char(subs(Hcloop));
    % Define 's' as transfer function variable
    s = tf('s');
    % Evaluate the expression
    eval(['TFH = ',TFstring]);
    figure (1)
    impulse(TFH);   %plot the impulse reponse
    figure(2)
    step(TFH)       %plot the step response
```

5.3 Arduino Code

The following is the file used to control the Rocky:

```
// Start the robot flat on the ground
// compile and load the code
// wait for code to load (look for "done uploading" in the Arduino IDE)
// wait for red LED to flash on board
// gently lift body of rocky to upright position
// this will enable the balancing algorithm
```

```
// wait for the buzzer
// let go
//
// The balancing algorithm is implemented in BalanceRocky()
// which you should modify to get the balancing to work
//


#include <Balboa32U4.h>
#include <Wire.h>
#include <LSM6.h>
#include "Balance.h"


extern int32_t angle_accum;
extern int32_t speedLeft;
extern int32_t driveLeft;
extern int32_t distanceRight;
extern int32_t speedRight;
extern int32_t distanceLeft;
extern int32_t distanceRight;
float speedCont = 0;
float displacement_m = 0;
int16_t limitCount = 0;
uint32_t cur_time = 0;
float distLeft_m;
float distRight_m;


extern uint32_t delta_ms;
float measured_speedL = 0;
float measured_speedR = 0;
float desSpeedL=0;
float desSpeedR =0;
float dist_accumL_m = 0;
float dist_accumR_m = 0;
float dist_accum = 0;
float speed_err_left = 0;
float speed_err_right = 0;
float speed_err_left_acc = 0;
float speed_err_right_acc = 0;
float errAccumRight_m = 0;
float errAccumLeft_m = 0;
float prevDistLeft_m = 0;
float prevDistRight_m = 0;
float angle_rad_diff = 0;
float angle_rad;                // this is the angle in radians
float angle_rad_accum = 0;      // this is the accumulated angle in radians
float angle_prev_rad = 0; // previous angle measurement
extern int32_t displacement;
int32_t prev_displacement=0;
uint32_t prev_time;

#define G_RATIO (162.5)



LSM6 imu;
Balboa32U4Motors motors;
Balboa32U4Encoders encoders;
Balboa32U4Buzzer buzzer;
Balboa32U4ButtonA buttonA;


```

```
#define FIXED_ANGLE_CORRECTION (0.26)  // ***** Replace the value 0.25 with the value you obtained from the Gyro calibration procedure




////////////////////////////////////////////////////////////////////////////////////////////
// This is the main function that performs the balancing
// It gets called approximately once every 10 ms  by the code in loop()
// You should make modifications to this function to perform your
// balancing
////////////////////////////////////////////////////////////////////////////////////////////

void BalanceRocky()
{

    // **************Enter the control parameters here

  float Kp =  5.5943e+03;
  float Ki = 2.4907e+04;
  float Jp = -2.0795e+03;
  float Ji = 145.3917;
  float Ci = -6.3347e+03;




    float v_c_L, v_c_R; // these are the control velocities to be sent to the motors
    float v_d = 0; // this is the desired speed produced by the angle controller


   // Variables available to you are:
   // angle_rad  - angle in radians
   // angle_rad_accum - integral of angle
   // measured_speedR - right wheel speed (m/s)
   // measured_speedL - left wheel speed (m/s)
   // distLeft_m - distance traveled by left wheel in meters
   // distRight_m - distance traveled by right wheel in meters  (this is the integral of the velocities)
   // dist_accum - integral of the distance

   // *** enter an equation for v_d in terms of the variables available ****
    v_d = Kp * angle_rad + Ki * angle_rad_accum;

  // The next two lines implement the feedback controller for the motor. Two separate velocities are calculated.
  //
  //
  // We use a trick here by criss-crossing the distance from left to right and
  // right to left. This helps ensure that the Left and Right motors are balanced

  // *** enter equations for input signals for v_c (left and right) in terms of the variables available ****
    v_c_L = v_d - (Jp * measured_speedL + Ji * distRight_m + Ci * dist_accum);
    v_c_R = v_d - (Jp * measured_speedR + Ji * distLeft_m + Cl * dist_accum);




    // save desired speed for debugging
    desSpeedL = v_c_L;
    desSpeedR = v_c_R;
```

```
    // the motor control signal has to be between +- 300. So clip the values to be within that range
    // here
    if(v_c_L > 300) v_c_L = 300;
    if(v_c_R > 300) v_c_R = 300;
    if(v_c_L < -300) v_c_L = -300;
    if(v_c_R < -300) v_c_R = -300;

    // Set the motor speeds
    motors.setSpeeds((int16_t) (v_c_L), (int16_t)(v_c_R));

}



void setup()
{
  // Uncomment these lines if your motors are reversed.
  // motors.flipLeftMotor(true);
  // motors.flipRightMotor(true);

  Serial.begin(9600);
  prev_time = 0;
  displacement = 0;
  ledYellow(0);
  ledRed(1);
  balanceSetup();
  ledRed(0);
  angle_accum = 0;

  ledGreen(0);
  ledYellow(0);
}



int16_t time_count = 0;
extern int16_t angle_prev;
int16_t start_flag = 0;
int16_t start_counter = 0;
void lyingDown();
extern bool isBalancingStatus;
extern bool balanceUpdateDelayedStatus;

void UpdateSensors()
{
  static uint16_t lastMillis;
  uint16_t ms = millis();

  // Perform the balance updates at 100 Hz.
  balanceUpdateDelayedStatus = ms - lastMillis > UPDATE_TIME_MS + 1;
  lastMillis = ms;

  // call functions to integrate encoders and gyros
  balanceUpdateSensors();

  if (imu.a.x < 0)
  {
    lyingDown();
    isBalancingStatus = false;
  }
  else
  {
    isBalancingStatus = true;
```

```
  }
}



void GetMotorAndAngleMeasurements()
{
    // convert distance calculation into meters
    // and integrate distance
    distLeft_m = ((float)distanceLeft)/((float)G_RATIO)/12.0*80.0/1000.0*3.14159;
    distRight_m = ((float)distanceRight)/((float)G_RATIO)/12.0*80.0/1000.0*3.14159;
    dist_accum += (distLeft_m+distRight_m)*0.01/2.0;

    // compute left and right wheel speed in meters/s
    measured_speedL = speedLeft/((float)G_RATIO)/12.0*80.0/1000.0*3.14159*100.0;
    measured_speedR = speedRight/((float)G_RATIO)/12.0*80.0/1000.0*3.14159*100.0;

    prevDistLeft_m = distLeft_m;
    prevDistRight_m = distRight_m;


    // this integrates the angle
    angle_rad_accum += angle_rad*0.01;
    // this is the derivative of the angle
    angle_rad_diff = (angle_rad-angle_prev_rad)/0.01;
    angle_prev_rad  = angle_rad;

}

void balanceResetAccumulators()
{
    errAccumLeft_m = 0.0;
    errAccumRight_m = 0.0;
    speed_err_left_acc = 0.0;
    speed_err_right_acc = 0.0;
}



void loop()
{
  static uint32_t prev_print_time = 0;   // this variable is to control how often we print on the serial monitor
  int16_t distanceDiff;    // this stores the difference in distance in encoder clicks that was traversed by the right vs the left wheel
  static float del_theta = 0;
  char enableLongTermGyroCorrection = 1;

  cur_time = millis();                   // get the current time in miliseconds


  if((cur_time - prev_time) > UPDATE_TIME_MS){
    UpdateSensors();                    // run the sensor updates.

    // calculate the angle in radians. The FIXED_ANGLE_CORRECTION term comes from the angle calibration procedure (separate sketch available for this)
    // del_theta corrects for long-term drift
    angle_rad = ((float)angle)/1000/180*3.14159 - FIXED_ANGLE_CORRECTION - del_theta;

    if(angle_rad > 0.1 || angle_rad < -0.1)      // If angle is not within +- 6 degrees, reset counter that waits for start
    {
      start_counter = 0;
   }

```

```
  if(angle_rad > -0.1 && angle_rad < 0.1 && ! start_flag)
  {
    // increment the start counter
    start_counter++;
    // If the start counter is greater than 30, this means that the angle has been within +- 6 degrees for 0.3 seconds, then set the start_flag
    if(start_counter > 30)
    {
      balanceResetEncoders();
      start_flag = 1;
      buzzer.playFrequency(DIV_BY_10 | 445, 1000, 15);
      Serial.println("Starting");
      ledYellow(1);
    }
  }


  // every UPDATE_TIME_MS, if the start_flag has been set, do the balancing
  if(start_flag)
  {
    GetMotorAndAngleMeasurements();
    if(enableLongTermGyroCorrection)
      del_theta = 0.999*del_theta + 0.001*angle_rad;  // assume that the robot is standing. Smooth out the angle to correct for long-term gyro drift

    // Control the robot
    BalanceRocky();
  }
  prev_time = cur_time;
  }
// if the robot is more than 45 degrees, shut down the motor
  if(start_flag && angle_rad > .78)
  {
    motors.setSpeeds(0,0);
    start_flag = 0;
  }
  else if(start_flag && angle < -0.78)
  {
    motors.setSpeeds(0,0);
    start_flag = 0;
  }

// kill switch
  if(buttonA.getSingleDebouncedPress())
  {
      motors.setSpeeds(0,0);
      while(!buttonA.getSingleDebouncedPress());
  }

if(cur_time - prev_print_time > 103)   // do the printing every 105 ms. Don't want to do it for an integer multiple of 10ms to not hog the processor
  {
        Serial.print(angle_rad);
        Serial.print("\t");
        Serial.print(distLeft_m);
        Serial.print("\t");
        Serial.print(measured_speedL);
        Serial.print("\t");
        Serial.print(measured_speedR);
        Serial.print("\t");
       Serial.println(speedCont);
       prev_print_time = cur_time;
  }

```

```

}
```